君正®

T23 SFC 开发指南

Date: 2023-05

北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

Copyright © 2005-2022 Ingenic Semiconductor Co. Ltd. All rights reserved.

No part of this document may be reproduced or transmitted in any form or by any means without prior written consent of Ingenic Semiconductor Co. Ltd.

Trademarks and Permissions

君正 Ingenic 、Ingenic and Ingenic icons are trademarks of Ingenic Semiconductor Co.Ltd. All other trademarks and trade names mentioned in this document are the property of their respective holders.

Disclaimer

All the deliverables and data in this folder serve only as a reference for customer development. Please read through this disclaimer carefully before you use the deliverables and data in this folder. You may use the deliverables in this folder or not. However, by using the deliverables and data in this folder, you agree to accept all the content in this disclaimer unconditional and irrevocable. If you do not find the content in this disclaimer reasonable, you shall not use the deliverables and data in this folder.

The deliverables and data in this folder are provided "AS IS" without representations, guarantees or warranties of any kind (either express or implied). To the maximum extent permitted by law, Ingenic Semiconductor Co., Ltd (Ingenic) provides the deliverables and data in this folder without implied representations, guarantees or warranties, including but not limited to implied representations, guarantees and warranties of merchantability, non-infringement, or fitness for a particular purpose. Deviation of the data provided in this folder may exist under different test environments.

Ingenic takes no liability or legal responsibility for any design and development error, incident, negligence, infringement, and loss (including but not limited to any direct, indirect, consequential, or incidental loss) caused by the use of data in this folder. Users shall be responsible for all risks and consequences caused by the use of data in this folder.

北京君正集成电路股份有限公司
地址：北京市海淀区西北旺东路 10 号院东区 14 号楼君正大厦
电话:**(86-10)56345000**
传真:**(86-10)56345001**
Http: //www.ingenic.cn

前言

概述

本文档主要介绍 lngenic T23 SPI flash 的添加以及配置。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
T23 SFC 开发指南	V1.0

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修订记录

修订记录累积了每次文档更新的说明。最新版本的文档包含以前所有文档版本的更新内容。

日期	版本	修订章节
2023-05	1.0	第一次正式版本发布

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

目录

1 添加新的 Nor flash 1
1.1 uboot 中添加 nor flash 配置 1
1.2 kernel 中添加 nor flash 配置 4
2 添加新的 Nand flash 7
2.1 SPl 中添加 nand flash 配置 7
2.2 uboot 中添加 nand flash 配置 11
2.3 Kernel 中添加 nand flash 配置 15
3 关于单线/四线配置 16
3.1 uboot 中配置单线/四线 16
3.2 kernel 中配置单线/四线 16
4 nor 与 nand 的选择及配置 17
4.1 uboot 中编译 nor 或者 nand 17
4.2 kernel 中配置选择 nor 或者 nand 17

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

1 添加新的 Nor Flash

一款新的 norflash 需要在 uboot 和 kernel 中同步添加。

1.1 uboot 中添加 nor flash 配置

步骤一：确认 ID

相同厂商制作的 flash 会有相同的厂商 ID，不同厂商制作的 flash 通常厂商 ID 会不同，添加 flash 前先确定是否已经添加了对应的厂商 ID。

以 norflash IS25LP128F 为例（默认读取 ID 的命令是 9F），其厂商 ID 为 9Dh，设备 ID 为 6018h。

```
$ vi drivers/mtd/spi/spi_flash.c
```

图 1-1 厂商 ID 示意图

Manufacturer ID		**(MF7-MF0)**	
ISSI Serial Flash		9Dh	
Instruction	**ABh**	**90h**	**9Fh**
Part Number	Device ID (ID7-ID0)		Memory Type + Capacity (ID15-ID0)
IS25LP128F	17h		6018h
IS25WP128F	17h		7018h

如下图所示，copy 一份现有的配置，将第二个参数改成厂商 ID，如 IS25LP128F 的厂商 ID 为 9d，故添加 0x9d 的配置；如果厂商 ID 已经存在，则跳过此步骤。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

图 1-2 添加厂商 ID 配置图

```
458 static const struct {
459     const u8 shift;
460     const u8 idcode;
461     struct spi_flash *(*probe) (struct spi_slave *spi, u8 *idcode);
462 } flashes[] = {
463     /* Keep it sorted by define name */
464 #ifdef CONFIG_SPI_FLASH_INGENIC
465 #ifdef CONFIG_SPI_FLASH_INGENIC_NAND
466     { 0, 0xc8, spi_flash_probe_ingenic_nand, },
467 #endif
468     { 0, 0x52, spi_flash_probe_ingenic, },
469     { 0, 0x5e, spi_flash_probe_ingenic, },
470     { 0, 0xe0, spi_flash_probe_ingenic, },
471     { 0, 0xc2, spi_flash_probe_ingenic, },
472     { 0, 0xef, spi_flash_probe_ingenic, },
473     { 0, 0x1c, spi_flash_probe_ingenic, },
474     { 0, 0xF8, spi_flash_probe_ingenic, },
475     { 0, 0x20, spi_flash_probe_ingenic, },
476     { 0, 0x68, spi_flash_probe_ingenic, },
477     { 0, 0xa1, spi_flash_probe_ingenic, },
478     { 0, 0x0b, spi_flash_probe_ingenic, },
479     { 0, 0x85, spi_flash_probe_ingenic, },
480     { 0, 0xba, spi_flash_probe_ingenic, },
481     { 0, 0x9d, spi_flash_probe_ingenic, },
482
~/work/isvp/work/opensource/uboot/drivers/mtd/spi/spi_flash.c [HEX=0][481,43,6
```

步骤二：添加 flash 属性参数

```
$ vi drivers/spi/jz_spi.h
```

找到 jz_spi_support_table 数组，按照已有的 flash 格式复制粘贴一份，主要修改 name, id_manufactory, sector_size, size。

以添加一个 16M 的 flash IS25LP128F 为例：

```
{
    .name          = "IS25LP128F",
    .id_manufactory = 0x9d6018,
    .page_size      = 256,
    .sector_size     = (64 * 1024),
    .addr_size = 3,
    .size          = (16 * 1024 * 1024),
    .quad_mode = {
        .dummy_byte = 8,
        .RDSR_CMD = CMD_RDSR,
        .WRSR_CMD = CMD_WRSR,
        .RDSR_DATE = 0x40,
        .RD_DATE_SIZE = 1,
        .WRSR_DATE = 0x40,
        .WD_DATE_SIZE = 1,
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
            .cmd_read = CMD_QUAD_READ,
#ifdef CONFIG_JZ_SFC
            .sfc_mode = TRAN_SPI_QUAD,
#endif
        },
    },
```

参数介绍：

1) name：spi nor flash 的名字
2) id_manufactory：spi flash 的 id, 厂家 ID+设备 ID，对于 IS25LP128F 即为 0x9d6018
3) page_size：spi flash 写操作的页大小
4) sector_size：是 flash 擦的大小（建议设置 64k，如果不支持请设成 32k）
5) addr_size：访问 flash 的地址长度(byte)，16MB 及以下为 3，大于 16MB 通常为 4
6) size：spi nor norflash 的总大小(byte)
7) quad_mode：四线模式，有需求可根据 spec 定义设置里面的参数
8) dummy_byte：dummy clocks 的个数（根据 spec 配置）
9) RDSR_CMD/WRSR_CMD：读写包含 QE 位状态寄存器的命令，如下图 RDSR_CMD 是 0x05,WRSR_CMD 是 0x01

RDSR	Read Status Register	SPI QPI	05h	Data out
WRSR	Write Status Register	SPI QPI	01h	Data in

10) RDSR_DATE/WDSR_DATE：是用来读/写 QE 位的值，如图 QE 是第六位，故为 0x40；
11) RD_DATE_SIZE/WD_DATE_SIZE：要操作的功能在寄存器中占了几位，如图，QE 只占 1bit，所以该值为 1；

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
	SRWD	QE	BP3	BP2	BP1	BP0	WEL	WIP
Default	0	0	0	0	0	0	0	0

12) cmd_read：四线模式快速读取数据的指令，以 IS25LP128F 为例，该命令为 0x6b；

FRQO	Fast Read Quad Output (3-byte Address)	SPI	6Bh	A <23:16>	A <15:8>	A <7:0>	Dummy(1) Byte	Quad Data out	

13) sfc_mode：对应 4 线模式控制器的传输值，默认为 TRAN_SPI_QUAD。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

图 1-3 jz_spi_support_table 数组内容图

```
    {
        .name           = "IS25LP128",
        .pagesize       = 256,
        .sectorsize     = 4 * 1024,
        .chipsize       = 16384 * 1024,
        .erasesize      = 32 * 1024,
        .id             = 0x9d6018,

        .block_info     = flash_block_info,
        .num_block_info = ARRAY_SIZE(flash_block_info),

        .addrsize       = 3,
        .pp_maxbusy     = 3,            /* 3ms */
        .se_maxbusy     = 400,          /* 400ms */
        .ce_maxbusy     = 8 * 10000,    /* 80s */

        .st_regnum      = 3,
#ifdef CONFIG_SPI_QUAD
        .quad_mode = &flash_quad_mode[1],
#endif
    },
```

注意

图上所有参数可在 flash spec 中查找。

1.2 kernel 中添加 nor flash 配置

```
$ vi arch/mips/xburst/soc-t23/chip-t23/isvp/common/spi_bus.c
```

按照下图 copy 一份现有的配置，修改参数即可：

参数介绍：

1) name：flash 的名称
2) pagesize：flash 的页大小
3) sectorsize：flash 块大小

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

4) chipsize：flash 总大小
5) erasesize：擦除大小（建议设置 64k，如果不支持请设成 32k）
6) id：flash 的 id, 厂家 ID+设备 ID
7) block_info：无效参数
8) num_block_info:无效参数
9) addrsize：访问 flash 的地址长度(byte)，16MB 及以下为 3，大于 16MB 通常为 4
10) pp_maxbusy：无效参数
11) se_maxbusy：无效参数
12) ce_maxbusy：无效参数
13) st_regnum：无效参数
14) quad_mode：配置 4 线模式参数，具体信息如下：

可以根据 flash 的手册选择对应的数组参数，如果数组中没有可以自行添加。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
struct spi_quad_mode  flash_quad_mode[] = {
    {
        .RDSR_CMD = CMD_RDSR_1,
        .WRSR_CMD = CMD_WRSR_1,
        .RDSR_DATE = 0x2,//the data is write the spi status register for QE bit
        .RD_DATE_SIZE = 1,
        .WRSR_DATE = 0x2,//this bit should be the flash QUAD mode enable
        .WD_DATE_SIZE = 1,
        .cmd_read = CMD_QUAD_READ,//
        .sfc_mode = TRAN_SPI_QUAD,
        .dummy_byte = 8,
    },
    {
        .RDSR_CMD = CMD_RDSR,
        .WRSR_CMD = CMD_WRSR,
        .RDSR_DATE = 0x40,//the data is write the spi status register for QE bit
        .RD_DATE_SIZE = 1,
        .WRSR_DATE = 0x40,//this bit should be the flash QUAD mode enable
        .WD_DATE_SIZE = 1,
        .cmd_read = CMD_QUAD_READ,
        .sfc_mode = TRAN_SPI_QUAD,
    },
```

RDSR_CMD/WRSR_CMD：表示读写含有 QE 位状态寄存器的命令，在这里 CMD_RDSR 表示 0x05，CMD_WRSR 表示 0x01。

a. RDSR_DATE/WRSR_DATE：表示 QE 的偏移位。
b. RD_DATE_SIZE：表示 QE 在状态寄存器中占了多少 bit。
c. cmd_read：表示 4 线模式的读命令，这里 CMD_QUAD_READ 表示 0x6b。
d. sfc_mode：对应 4 线模式控制器的传输值，默认为 TRAN_SPI_QUAD。

注意

图上所有参数可在 flash spec 中查找。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2 添加新的 Nand Flash

2.1 SPl 中添加 nand flash 配置

步骤一：确认 ID

添加的目录为“tools/ingenic-tools/nand_device/”，首先确认该 NAND 的厂商 ID 是否已经支持，当该 NAND 的厂商 ID 不支持（以 dosilicon 东芯 nand 为例），拷贝一份已有 nand 配置文件(gd_nand.c)，修改其中一些参数内容。

```
$ cp gd_nand.c dosilicon_nand.c
$ vi dosilicon_nand.c
```

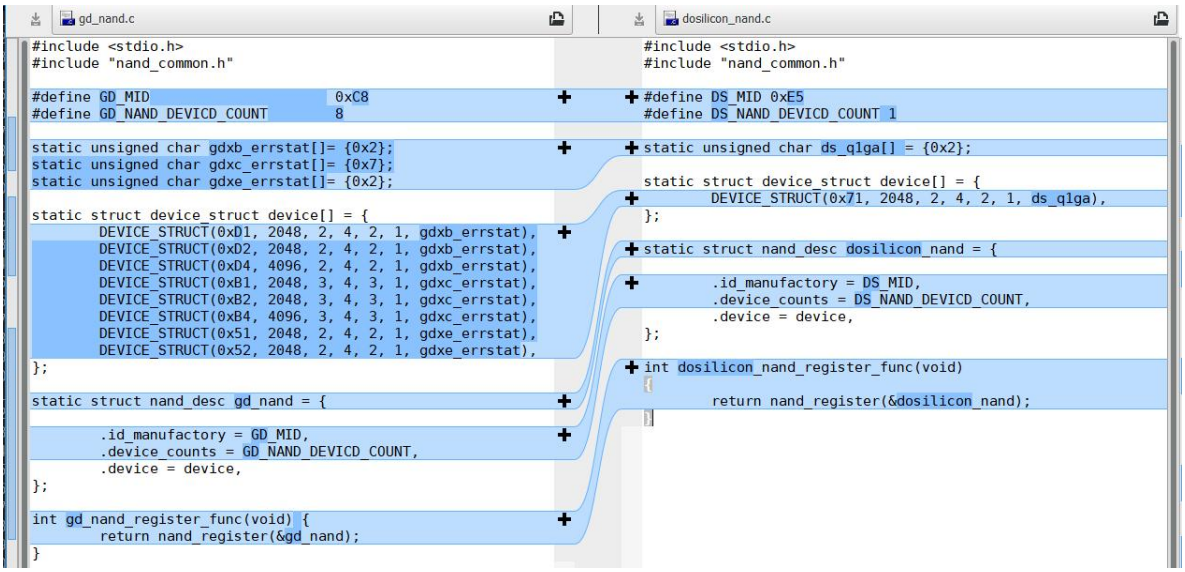
图 2-1　gc_nand.c 内容图

```
1  #include <stdio.h>
2  #include "nand_common.h"
3
4  #define GD_MID              0xC8
5  #define GD_NAND_DEVICD_COUNT        8
6
7  static unsigned char gdxb_errstat[]= {0x2};
8  static unsigned char gdxc_errstat[]= {0x7};
9  static unsigned char gdxe_errstat[]= {0x2};
10
11 static struct device_struct device[] = {
12     DEVICE_STRUCT(0xD1, 2048, 2, 4, 2, 1, gdxb_errstat),
13     DEVICE_STRUCT(0xD2, 2048, 2, 4, 2, 1, gdxb_errstat),
14     DEVICE_STRUCT(0xD4, 4096, 2, 4, 2, 1, gdxb_errstat),
15     DEVICE_STRUCT(0xB1, 2048, 3, 4, 3, 1, gdxc_errstat),
16     DEVICE_STRUCT(0xB2, 2048, 3, 4, 3, 1, gdxc_errstat),
17     DEVICE_STRUCT(0xB4, 4096, 3, 4, 3, 1, gdxc_errstat),
18     DEVICE_STRUCT(0x51, 2048, 2, 4, 2, 1, gdxe_errstat),
19     DEVICE_STRUCT(0x52, 2048, 2, 4, 2, 1, gdxe_errstat),
20 };
21
22 static struct nand_desc gd_nand = {
23
24     .id_manufactory = GD_MID,
25     .device_counts = GD_NAND_DEVICD_COUNT,
26     .device = device,
27 };
28
29 int gd_nand_register_func(void) {
30     return nand_register(&gd_nand);
31 }
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

需要修改的内容如下图：

图 2-2 修改内容对比图



参数介绍：

1) DS_MID：厂商 ID

DENSITY	VCC	1st Manufactory ID	2nd Device ID
1Gbit	3.3V	E5h	71h
	1.8V	E5h	21h

2) DS_NAND_DEVICD_COUNT：表示支持该系列 nand 的个数（相同厂商 ID，但是 deviceID 不同）

3) ds_q1ga：表示发生 ECC 错误，且不可恢复

SR[5:4] (ECC_S1, ECC_S0)	ECC Status	The bit shows the status of ECC as below: 00b = 0 bit error 01b = 1 ~ 4 bits error and been corrected. 10b = More than 4-bit error and not corrected. 11b = Reserved The value of ECC_Sx (S1:S0) bits will be clear as "00b" by Reset command or at the start of the Read operation. After a valid Read operation completion, the bit will be updated to reflect the ECC status of the current valid Read operation. The ECC_Sx (S1:S0) value reflects the ECC status of the content of the page 0 of block 0 after a power-on reset. If the internal ECC is disabled by the Set feature command, the ECC_Sx(S1:S0) are invalid.

4) DEVICE_STRUCT(0x71, 2048, 2, 4, 2, 1, ds_q1ga)

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
#define DEVICE_STRUCT(id, pagesize, addrlen, bit, bitcounts,  eccstatcount, err) {  \
        .device_id = id,    \
        .page_size = pagesize,  \
        .addr_len = addrlen,    \
        .ecc_bit = bit, \
        .bit_counts = bitcounts,    \
        .eccstat_count = eccstatcount, \
        .eccerrstatus = err,    \
}
```

a. device_id：设备 ID

b. page_size：页大小

c. addr_len：读写数据的地址长度

d. ecc_bit：状态寄存器中 ECC 的最低偏移位

e. bit_counts：状态寄存器中 ECC 所占 bit 数

f. eccstat_count：ECC 错误状态的个数

g. eccerrstatus：储存 ECC error status 内容的地址（通常是无法恢复或者意料之外的）

注意

如果厂商 ID 已经支持，只需要在该文件中添加该设备的参数即可。

步骤二：2G NAND 可能会涉及 plane 选择（如果没有可跳过此步骤）

示例：芯天下 XT26G02E

```
column |= (((page >> 6) & 1) << 12);
```

0：plane 表示偶数 block

1：plane 表示奇数 block

```
 if(curr_device->device_id == 0x20 || curr_device->device_id == 0x22)/*MXIC 2G plane select*/
     column |= (((page >> 6) & 1) << 12);
ndef CONFIG_SPI_STANDARD
 SFC_SEND_COMMAND(&sfc, CMD_FR_CACHE_QUAD, len, column, curr_device->addrlen, 8, 1, 0);
lse
 SFC_SEND_COMMAND(&sfc, CMD_FR_CACHE, len, column, curr_device->addrlen, 8, 1, 0);
ndif
 sfc_read_data((unsigned int *)dst_addr, len);

 if (!oob_flag && !(page % CONFIG_SPI_NAND_PPB)) {
     oob_flag = 1;
     goto read_oob;
 } else if (oob_flag) {
l/tyu/isvp/opensource/uboot/common/spl/spl_sfc_nand.c[POS=193,4][46%]29/05/21 - 11:49
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

Figure1. Array Organization

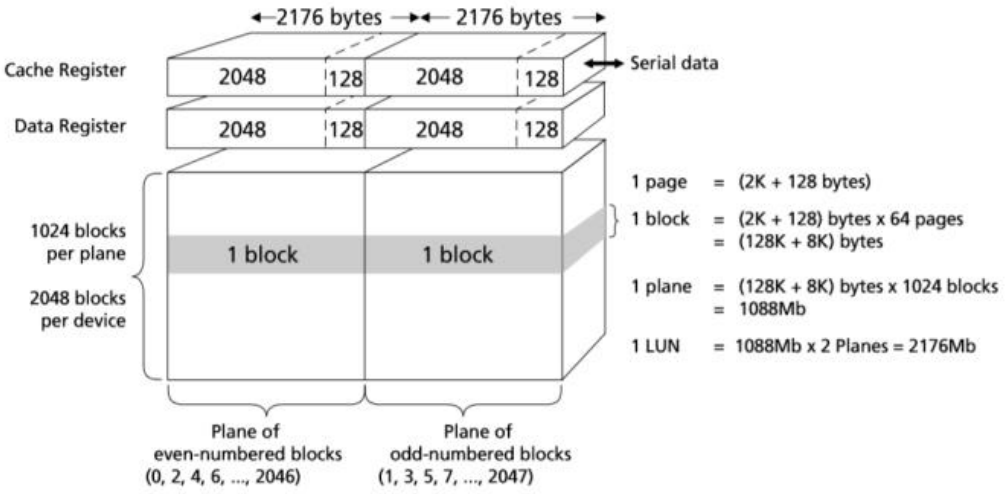
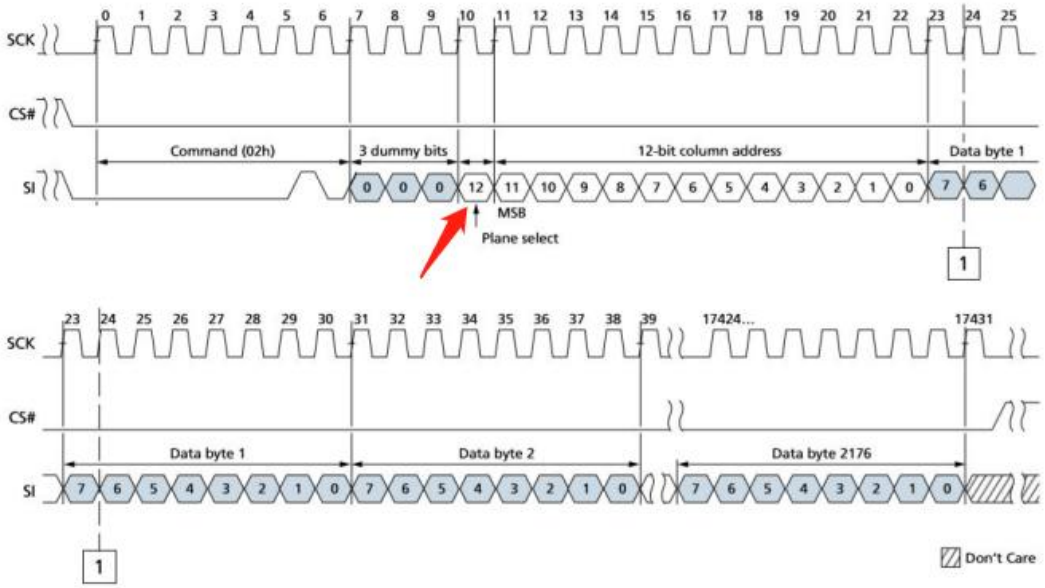


Figure14. PROGRAM LOAD (02h) Timing



Note: 1. Plane select is a dummy bit in 1Gb device.

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

2.2 uboot 中添加 nand flash 配置

步骤一：添加配置文件

在 “drivers/mtd/devices/nand_device/”这个目录下创建相应 flash 厂商的 c 文件。（例如： foresee_nand.c， 添加对江波龙 spinand 的支持）

```
$ cd drivers/mtd/devices/nand_device/
```

Copy 任意一个已支持厂商文件的内容到新添加 flash 文件中，并将新文件中定义的所有描述符名称进行修改.

```
$ cp xtx_nand.c foresee_nand.c
```

步骤二：修改参数

1. 对文件标红的地方进行专门修改：（以 foresee_nand.c 为例）

   A 处标红：支持此 flash 厂商的 device 数目，目前只支持江波龙的 FS35ND01G 这一款设备，故该值为 1，如果后面添加了江波龙厂商的其他 nand 设备，这里也要同步修改。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

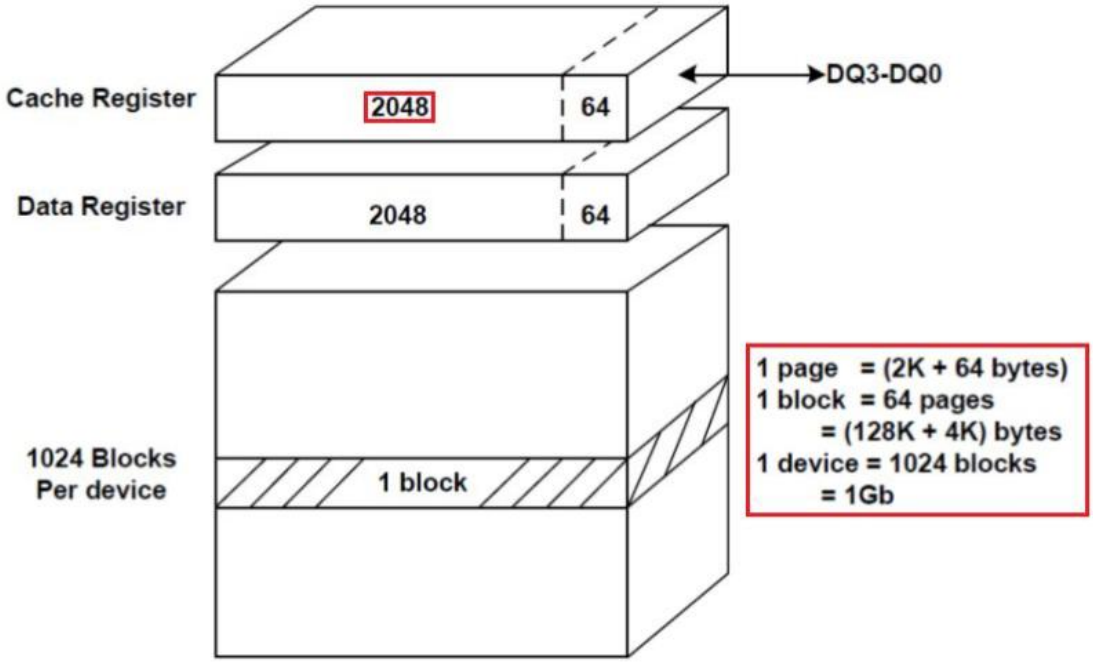
```
#include <malloc.h>
#include <linux/mtd/partitions.h>
#include "../jz_sfc_nand.h"
#include "nand_common.h"

#define FS_DEVICES_NUM        A. 1
#define TSETUP          B.    5
#define THOLD                 5
#define TSHSL_R               20
#define TSHSL_W               20

static struct jz_nand_base_param fs_param[FS_DEVICES_NUM] = {

        [0] = {
                /*FS35ND01G*/
                .pagesize = 2 * 1024,
            C.  .blocksize = 2 * 1024 * 64,
                .oobsize = 64,
                .flashsize = 2 * 1024 * 64 * 1024,

                .tSETUP  = TSETUP,
                .tHOLD   = THOLD,
                .tSHSL_R = TSHSL_R,
                .tSHSL_W = TSHSL_W,

                .ecc_max = 0x4,
                .need_quad = 1,
        },

};

static struct device_id struct device_id[FS_DEVICES_NUM] = {
      D. DEVICE_ID_STRUCT(0xA1, "FS35ND01G", &fs_param[0]),
};
```

B 处标红： 当前支持的 flash device 的 CS 时序（添加需要查看 device 数据手册， 如下图位置）

tSLCH	CS# Active Setup Time	5			ns
tCHSH	CS# Active Hold Time	5			ns
tSHCH	CS# Not Active Setup Time	5			ns
tCHSL	CS# Not Active Hold Time	5			ns
tSHSL/tCS	CS# High Time	20			ns

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

a. TSETUP(tSLCH)：CS# Active Setup Time
b. THOLD (tCHSH)：CS# Active Hold Time
c. TSHSL_R：CS #Deselect Time (for Array Read -> ArrayRead)
d. TSHSL_W：CS #Deselect Time (for Erase, Program or Read Status Registers ->Read Status Registers)



C 处标红：当前支持的 flash device 的参数信息填充。（添加需要查看 device 手册如下图位置）。

a. pagesize：device page size without oob size
b. blocksize：device page size without oob size
c. oobsize：oob area size in each page
d. flashsize：chip size without oob size
e. ecc_max：flash 可以检测和纠正的 ECC 最大位数。
f. need_quad：使能四线传输

D 处标红：添加支持的每款 device 信息（按照以下结构体进行填充）。

```
#define DEVICE_ID_STRUCT(id, name_string, parameter)
{\
    .id_device = id,          \
    .name = name_string,      \
    .param = parameter,       \}
struct device_id_struct {
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
    uint8_t id_device;                  //device id
    char *name;                         //device name
    struct jz_nand_base_param *param;   //与 C 处标红相对应的参数地址。
};
```

```
static int fs_nand_init(void) {
        struct jz_nand_device *fs_nand;
        fs_nand = kzalloc(sizeof(*fs_nand), GFP_KERNEL);
        if(!fs_nand) {
                pr_err("alloc fs_nand struct fail\n");
                return -ENOMEM;
        }

        fs_nand->id_manufactory = 0xCD;
        fs_nand->id_device_list = device_id;
        fs_nand->id_device_count = FS_DEVICES_NUM;

        fs_nand->ops.nand_read_ops.get_feature = fs_get_read_feature;
        return jz_spinand_register(fs_nand);
}
SPINAND_MOUDLE_INIT(fs_nand_init);
```

A 处标红：填充 flash 的厂商 ID

B 处标红：此处为必填，必须填充***_get_read_feature 函数地址。（主要功能是对 ECC value 进行处理，即***_get_read_feature 函数必须在此文件中实现，具体实现可参考其他 flash 文件的实现）。

C 处标红：填充当前 flash 厂商的特殊用法。主要是将 device 数据手册中的使用方法与 nand_common.c 中罗列地进行对比，把特殊用法编写成相应的函数，然后将函数地址填充于此。

步骤三：修改 Makefile

修改 Makefile 文件，将新添加的 flash 文件添加进去，进行编译。（以 foresee_nand.c 为例）

```
$ vi drivers/mtd/devices/Makefile
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

图 2-3 Makefile 修改图

```
LIB := $(obj)libsfc.o

COBJS-$(CONFIG_SFC_NOR) += jz_sfc_nor.o
COBJS-$(CONFIG_MTD_SFCNAND) += jz_sfc_common.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/ato_nand.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/gd_nand.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/mxic_nand.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/nand_common.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/winbond_nand.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/xtx_nand.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/foresee_nand.o
COBJS-$(CONFIG_MTD_SFCNAND) +=nand_device/dosilicon_nand.o
COBJS-$(CONFIG_MTD_SFCNAND) +=jz_sfc_nand.o
```

```
obj-y += ato_nand.o gd_nand.o mxic_nand.o xtx_nand.o dosilicon_nand.o foresee_nand.o xtx_mid0b_nand.o zetta_nand.o
obj-y += nand_common.o
```

2.3 Kernel 中添加 nand flash 配置

步骤一：添加配置文件

```
$ cd drivers/mtd/devices/nand_device/
```

如果此目录下没有该厂商 ID 的配置文件，拷贝一份现有的 nand 配置文件进行修改，步骤同 2.2. uboot 中添加 nand flash 配置 步骤一、步骤二类似。

步骤二：修改 Makefile

```
$ vi drivers/mtd/devices/nand_device/Makefile
```

添加编译此文件（以 foresee_nand.c 为例）。

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

3 关于单线/四线配置

3.1 uboot 中配置单线/四线

uboot 中默认配置单线模式。

```
$ vi include/configs/isvp_t23.h
```

查看是否开启了 CONFIG_SPI_QUAD 的宏配置，如果没有开启表示默认单线模式，开启表示 uboot 使用四线模式。

3.2 kernel 中配置单线/四线

在 kernel 目录下进行配置：

```
$ make menuconfig
```

通过 make menuconfig 进行配置：

```
Device Drivers  --->
    <*> Memory Technology Device (MTD) support  --->
        Self-contained MTD device drivers  --->
            sfc Mode (standard spi mode)  --->
```

单线：standard spi mode

四线：quad spi mode

配置完成可以通过.config 查看：

单线：

```
CONFIG_SPI_STARDARD=y
# CONFIG_SPI_QUAD is not set
```

四线：

```
# CONFIG_SPI_STARDARD is not set
CONFIG_SPI_QUAD=y
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

4 nor 与 nand 的选择及配置

4.1 uboot 中编译 nor 或者 nand

这里以 T23N 为例：

编译选择 NOR ：

```
$ make isvp_t23_sfcnor
```

编译选择 NAND：

```
$ make isvp_t23_sfcnand
```

注意

通常编译选项中会有关键字 nand（或者 nor）。

4.2 kernel 中配置选择 nor 或者 nand

通过 make menuconfig（这里以 T23 为例）进行配置：

```
Device Drivers  --->
    <*> Memory Technology Device (MTD) support  --->
        Self-contained MTD device drivers  --->
            <*>   Ingenic JZ SFC nor
            < >   Ingenic JZ SFC nand
```

NOR：Ingenic JZ SFC nor

NAND：Ingenic JZ SFC nand

配置完成可以通过.config 查看：

NOR：

```
CONFIG_MTD_JZ_SFC_NOR=y
# CONFIG_MTD_JZ_SFC_NAND is not set
```

NAND：

Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.

```
# CONFIG_MTD_JZ_SFC_NOR is not set
CONFIG_MTD_JZ_SFC_NAND=y
```


Copyright® 2005-2022 Ingenic Semiconductor Co., Ltd. All rights reserved.